

Universidad del Cauca

Facultad de ingeniería electrónica y telecomunicaciones

Microproyecto Aplicaciones Móviles

Tecnología en Telemática
Jose Manuel Perafan
Nelson Stiven Sandoval

Aplicaciones Móviles
PhD. Cristhian Nicolas Figueroa

Popayán, Cauca

2026

1. Enlace al Repositorio GIT

En este apartado se comparte el repositorio donde está guardado el código fuente de Gasti para iOS. La idea es que el docente pueda entrar, revisar los archivos y conocer cómo está organizado el proyecto.

https://github.com/Stivensan2025/Gasti-Version-IOS.git

2. Capturas de Pantalla

Estas imágenes muestran las pantallas principales de Gast.

2.1. Inicio

En esta pantalla se muestra un resumen de las finanzas: saldo actual, ingresos, gastos y movimientos recientes.

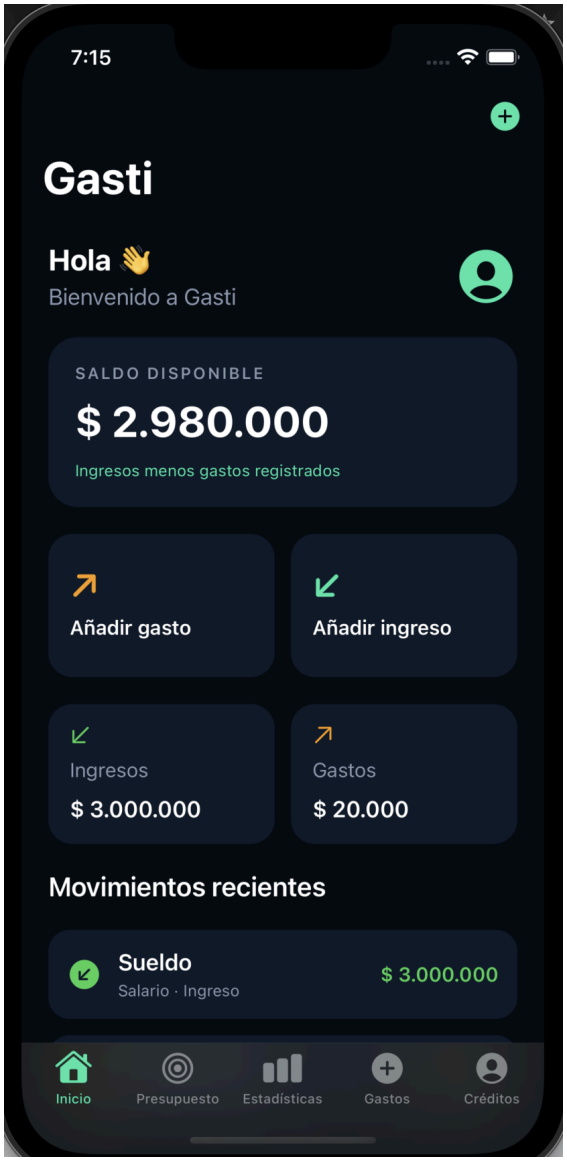


Figura: pantalla de Gasti para iOS.

2.2. Gastos

Aquí se puede consultar el total gastado y revisar los movimientos organizados en un historial. También aparece la opción para registrar un nuevo gasto.

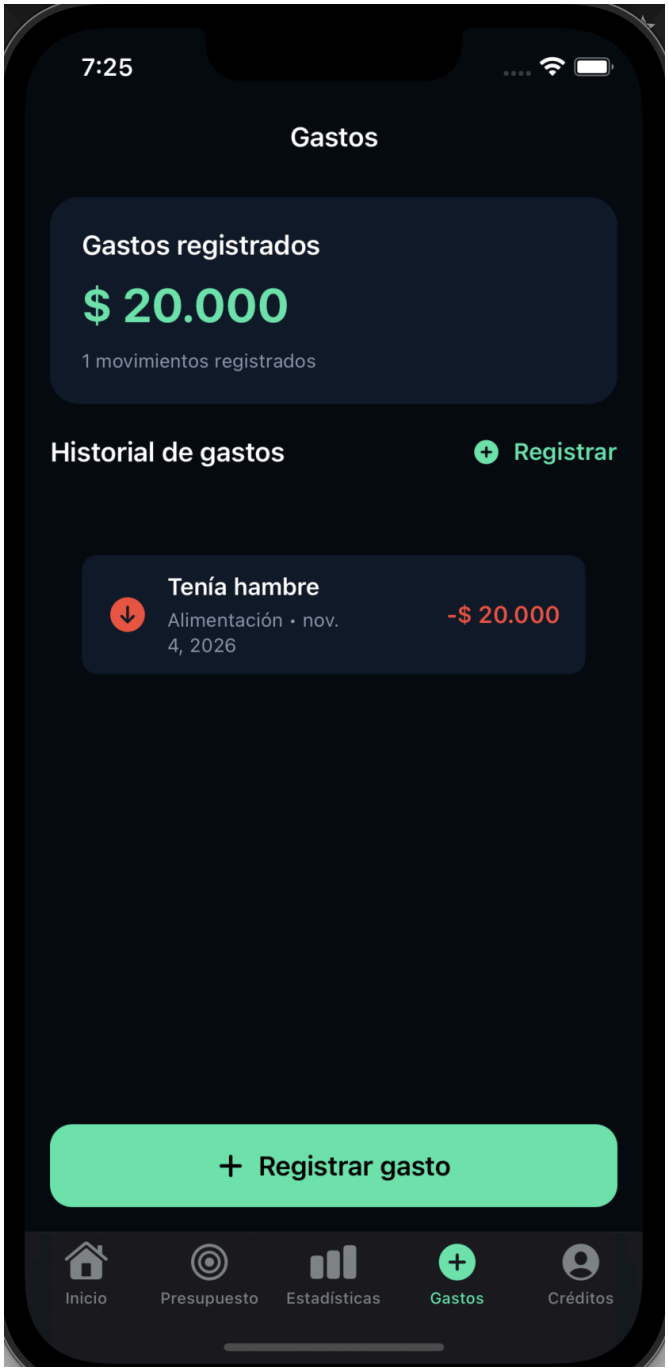


Figura: pantalla de Gasti para iOS.

2.3. Presupuesto

Esta sección permite visualizar el presupuesto mensual, cuánto se ha gastado y el avance por categorías.



Figura: pantalla de Gasti para iOS.

2.4. Estadísticas

La pantalla presenta gráficas y porcentajes para entender mejor cómo se distribuyen los gastos y comparar ingresos frente a gastos.



Figura: pantalla de Gasti para iOS.

2.5. Perfil

En el perfil aparece la información del usuario y un resumen de su actividad dentro de la aplicación.



Figura: pantalla de Gasti para iOS.

2.6. Login

En el apartado de login el usuario puede iniciar sesión o crear una cuenta. Este apartado permite al usuario tener un seguridad previa de sus gastos e ingresos.



Figura: pantalla de Gasti para iOS.

3. Descripción de la Arquitectura y Componentes

3.1. Diagrama de arquitectura

Para explicar cómo está organizada la aplicación, se realizó un diagrama a mano. En él se separan las vistas que utiliza la persona, la parte encargada de manejar la lógica y los servicios de datos. También se muestra la relación de la aplicación con Supabase y con el almacenamiento local.

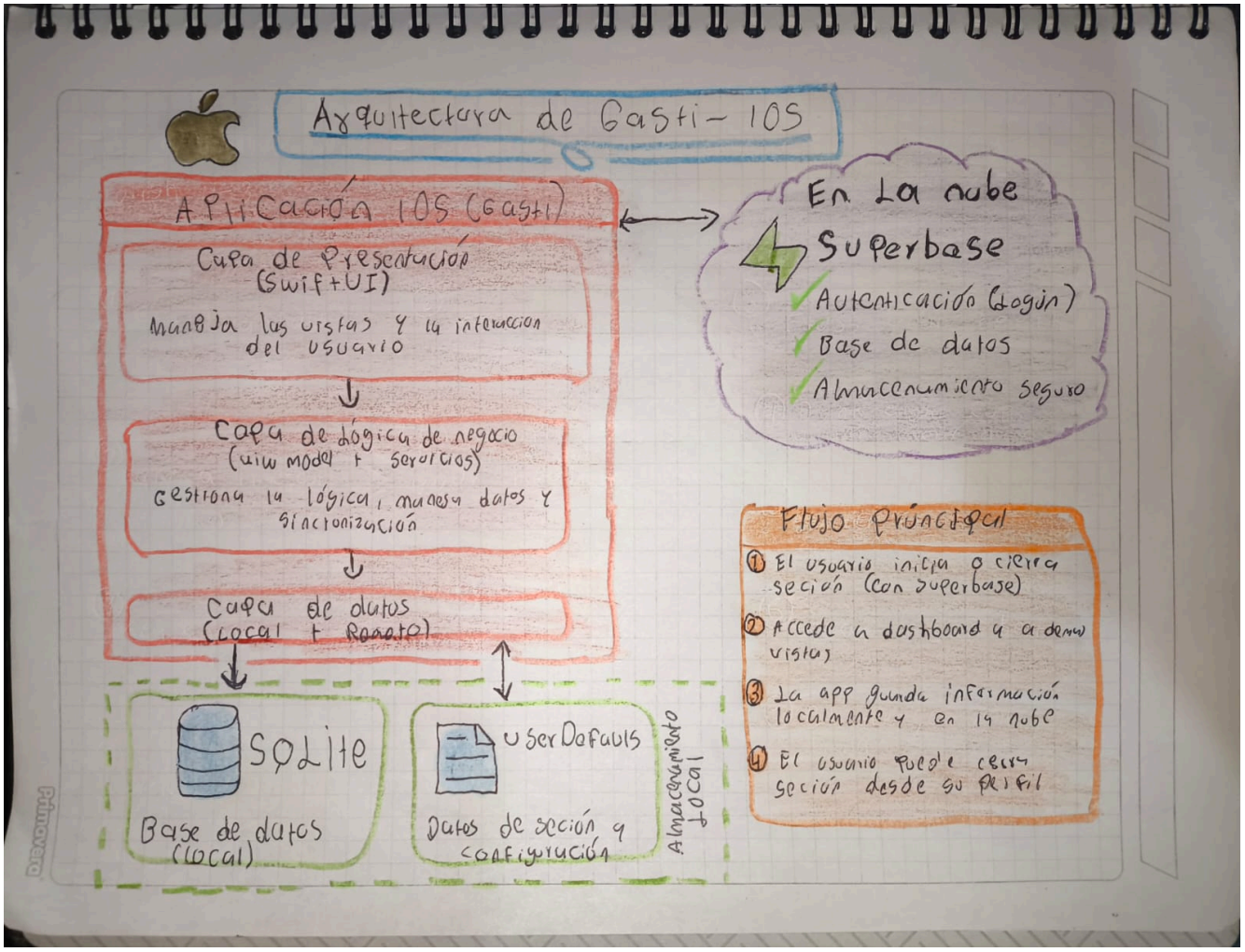


Figura: diagrama de arquitectura de Gasti para iOS, elaborado a mano.

Una aclaración importante: el dibujo representa la arquitectura planteada para Gasti, pero no todos sus elementos quedaron comprobados en la revisión del código. En el resumen técnico revisado se identificó UserDefaults como almacenamiento local de transacciones y presupuestos. En cambio, no quedó confirmado el uso de SQLite ni la sincronización de esos movimientos con tablas remotas de Supabase. Por eso, esos dos puntos se explican como elementos del diagrama que todavía necesitan validación, y no como funciones completamente verificadas.

3.2. Implementación de ViewModel

SwiftUI permite construir las pantallas de forma declarativa: se define cómo debe verse cada vista y esta responde a los cambios de estado de la aplicación. En una estructura que utiliza ViewModel, este componente se encarga de conectar la interfaz con la lógica, organizar los datos que necesita cada pantalla y actualizar la información cuando se realizan acciones, como registrar un gasto o consultar un presupuesto.

En la documentación técnica que se revisó no se confirmó cuáles clases ViewModel están implementadas ni sus nombres. Por eso, para no afirmar algo que no se ha comprobado, esta sección describe el papel que tendría el ViewModel en la arquitectura. Antes de entregar, se recomienda revisar el proyecto en Xcode y mencionar las clases concretas, si existen. Si no se utiliza ese patrón, se debe explicar cómo se manejan actualmente los estados desde las vistas y los servicios.

3.3. Consumo de APIs (Networking)

En el proyecto se identificó el archivo APIService.swift, que se preparó para realizar consultas a un servicio externo. También aparece APIView.swift, una vista que servía para probar esa consulta. Durante la organización de la navegación se propuso reemplazar la pestaña API por la sección de Gastos, así que es necesario comprobar si el servicio externo sigue utilizándose en la versión final o si quedó solamente como parte de las pruebas.

Por otro lado, Gasti utiliza Supabase para la autenticación de usuarios. El archivo SupabaseService.swift centraliza esa conexión. La revisión disponible confirma su uso para autenticación, pero no permite asegurar que los gastos y presupuestos ya se están enviando y sincronizando con una base de datos remota.

3.4. Persistencia de datos

Para conservar información entre usos de la aplicación, se definió PersistenceService.swift., este servicio guarda localmente transacciones y presupuestos mediante UserDefaults. Esto permite que ciertos datos permanezcan guardados en el dispositivo y puedan consultarse después.

También se definieron dos modelos principales: Transaction.swift, que representa los movimientos y diferencia entre ingresos y gastos, y Budget.swift, que contiene la información de los presupuestos, como el nombre, el monto y el mes.

Supabase se utiliza para la autenticación. Sin embargo, la sincronización de transacciones y presupuestos con tablas remotas, así como las reglas de seguridad RLS, quedaron pendientes de verificación en el resumen técnico. Por eso, se puede hablar de almacenamiento local identificado y de autenticación con Supabase, pero no asegurar todavía una sincronización completa en la nube.

4. Reflexión Comparativa: SwiftUI frente a Jetpack Compose

Al trabajar en Gasti para iOS y compararlo con el desarrollo en Android, se nota que SwiftUI y Jetpack Compose tienen una idea de trabajo bastante parecida: las dos tecnologías permiten construir las pantallas de manera declarativa. Es decir, uno organiza los componentes de la interfaz y define cómo deben reaccionar cuando cambian los datos.

La diferencia principal está en las herramientas y el lenguaje que utiliza cada plataforma. SwiftUI trabaja con Swift y se desarrolla normalmente desde Xcode para dispositivos Apple. Jetpack Compose utiliza Kotlin y se integra con Android Studio. Aunque algunos conceptos se parecen, la forma de escribir el código, configurar el proyecto y conectarse con los servicios de cada sistema cambia.

En nuestra experiencia, desarrollar para iOS fue una oportunidad para conocer otro entorno y entender que no basta con trasladar exactamente el código de Android. Hay que adaptar la aplicación a las herramientas y características de cada plataforma. Aun así, los conocimientos de organización de pantallas, estados y componentes que se usan en Jetpack Compose sirven como una buena base para aprender SwiftUI.